

Exercise 3

1. Large Language Models Used

To evaluate the efficacy of Generative AI in higher education, the following models were tested, specifically leveraging their advanced reasoning and search capabilities:

- **DeepSeek-R1:** DeepThink Mode (Explicit Chain-of-Thought reasoning)
- **Qwen:** Thinking Mode (Pedagogical and structural reasoning)
- **Grok 4.5:** DeepSearch (Real-time web synthesis and industry context)

Model 1: DeepSeek-R1 (DeepThink Mode)

REPORT: USING GENERATIVE AI FOR UNIVERSITY STUDIES

Focus Subjects: Probability and Statistics, Database Management Systems (DBMS), and Software Engineering (SE)

Date: August 2026

Purpose: To provide a concise, actionable strategy for integrating Generative AI into technical coursework while maintaining academic integrity.

EXECUTIVE SUMMARY

Generative AI (e.g., ChatGPT, Claude, Gemini) serves as a powerful cognitive partner for university students. In technical subjects, it functions as a 24/7 tutor, coding mentor, and system architect. However, success depends on using AI to enhance your thinking, not replace it. This report outlines subject-specific tactics and ethical guardrails for effective use.

1. PROBABILITY AND STATISTICS

The core challenge of statistics is moving from abstract formulas to intuitive understanding. GenAI can bridge this gap effectively.

- **Simulation and Visualization:** Prompt the AI to generate Python (Matplotlib) or R scripts to simulate distributions (e.g., rolling dice or the Central Limit Theorem). Watching the data come to life graphically solidifies theoretical concepts faster than static textbook diagrams.

- Conceptual Translation (The Feynman Technique): Ask the AI to explain complex theorems (like Maximum Likelihood Estimation or Bayes' Rule) using relatable analogies. Then, paste your own explanation and ask the AI to act as a strict professor, identifying logical gaps or mathematical imprecisions in your reasoning.
- Interpreting Outputs: Paste regression tables or ANOVA results from SPSS/R into the AI and ask for a plain-English translation suitable for a research paper's "Results" section.

2. DATABASE MANAGEMENT SYSTEMS (DBMS)

DBMS requires logical rigor, set theory, and syntactic precision with SQL. GenAI excels here due to the structured nature of the data.

- Step-by-Step Normalization: Provide the AI with a messy, denormalized table (e.g., containing Student, Course, Instructor data). Ask it to walk you through the normalization process from 1NF to 3NF (or BCNF), identifying functional dependencies and explaining the reasons for every table split.
- SQL Query Optimization: Do not just ask for a query. Write your own, then paste it and ask the AI to critique its performance. Request specific suggestions for indexes (B-tree vs. Hash) and ask for a rewrite using advanced techniques like Window Functions or CTEs. Always ask for a side-by-side performance comparison.
- Concurrency Scenarios: Ask the AI to generate specific SQL transaction sequences that result in deadlocks or non-repeatable reads, then have it explain how isolation levels (READ COMMITTED vs. SERIALIZABLE) prevent these phenomena.

3. SOFTWARE ENGINEERING (SE)

Software Engineering extends beyond writing code to encompass design, architecture, testing, and lifecycle management.

- System Design and Trade-offs: Prompt the AI to design a high-level system (e.g., a URL shortener or e-commerce platform). Request it to explicitly compare architectural choices, such as SQL vs. NoSQL, or Monolithic vs. Microservices, including the specific pros and cons for your use case.
- Refactoring and Design Patterns: Feed the AI "spaghetti code" and ask it to refactor the code using SOLID principles. Specifically request the implementation of relevant design patterns (e.g., Factory, Strategy, Observer) to make the code more maintainable. Always ask the AI to explain why the new structure is superior.

- **Automated Testing:** Describe a function specification and ask the AI to generate unit tests (using JUnit or PyTest) *before* writing the main code. This enforces a Test-Driven Development (TDD) mindset. You can also ask the AI to generate CI/CD YAML scripts for automating builds and tests.

4. UNIVERSAL PROMPT STRATEGIES (CROSS-CUTTING)

To maximize efficiency across all three subjects, incorporate these generic prompt techniques:

- **The Socratic Tutor:** Instruct the AI to guide you with questions rather than giving the final answer straight away. This forces active problem-solving.
- **The Knowledge Compressor:** Paste 15 pages of lecture notes and request a singlepage cheat sheet organized by key topics, definitions, and formulas for quick revision before exams.
- **The Debugger:** Provide error messages and stack traces, and ask the AI to hypothesize three possible root causes along with diagnostic steps to identify the actual one.

5. CRITICAL ETHICAL CONSIDERATIONS AND LIMITATIONS

Using GenAI requires strict discipline to avoid academic misconduct and cognitive atrophy.

- **Plagiarism and Detection:** Never submit raw AI-generated text or code as your own. Use AI for drafts and outlines, but rewrite the final submission in your own words. Universities actively use detection tools like Turnitin.
- **The Hallucination Problem:** AI will confidently produce incorrect mathematical derivations or suggest SQL functions that do not exist in your specific database version. Treat the AI as a fallible intern; always cross-check critical outputs with official documentation (e.g., Python docs, Oracle/MySQL manuals).
- **Erosion of Core Skills:** Over-reliance on AI for syntax or query writing will hurt you in closed-book exams. Implement a hybrid rule: always attempt the problem yourself with pen and paper first, then use AI to verify your logic or to get a hint when completely stuck.

CONCLUSION

Generative AI is an accelerator, not a substitute. For Probability and Statistics, it builds intuition; for DBMS, it sharpens logical design; for Software Engineering, it elevates architectural thinking. By applying these targeted strategies and maintaining strict oversight, students can dramatically improve their comprehension, coding efficiency, and exam readiness while preserving their academic integrity.

Model 2: Qwen3.7-Plus (Thinking Mode)

Report: Using Generative AI to Master University STEM & CS Subjects

Executive Summary

For university students, Generative AI is most effective when used as a Socratic tutor and collaborative partner, not an answer generator. By engaging in iterative, guided dialogue, students can accelerate their understanding of complex technical subjects while maintaining academic integrity and building genuine competence.

Subject-Specific Strategies

1. Probability and Statistics

- **Conceptual Analogies:** Ask the AI to explain abstract concepts (e.g., p-values, Central Limit Theorem) using real-world, relatable analogies.
- **Logic Checking:** Instead of asking for the final answer, provide your step-by-step thought process and ask the AI to identify where your logic might be flawed.
- **Practice Generation:** Prompt the AI to generate custom practice problems based on your weak areas, and have it grade your attempts with detailed feedback.

2. Database Management Systems (DBMS)

- **Guided Normalization:** Provide a raw table schema and functional dependencies, and ask the AI to guide you *step-by-step* through normalizing it to 3NF or BCNF, rather than just giving the final answer.
- **SQL Optimization:** Paste your working but slow SQL queries and ask the AI to explain how to optimize them using indexing or window functions, detailing the "why" behind the performance gains.

- Scenario Simulation: Ask the AI to simulate concurrency scenarios (e.g., two transactions updating the same record) to visualize how ACID properties and locking mechanisms work in practice.

3. Software Engineering

- Architectural Trade-offs: Provide a project scope (e.g., "food delivery app backend") and ask the AI to compare architectural choices (Monolith vs. Microservices), focusing on real-world trade-offs.
- Code Reviews: Submit your own code and ask the AI to critique it against industry standards like SOLID principles or DRY (Don't Repeat Yourself), prompting *you* to refactor it.
- Test-Driven Development (TDD): Before writing a function, ask the AI to act as a QA engineer and generate a comprehensive list of edge cases and unit test scenarios you need to cover.

The 4 Golden Rules of AI-Assisted Learning

1. Guide, Don't Tell: Always prompt the AI to ask you questions or explain the *process*, never just to give the final answer.
2. Verify Everything: GenAI can hallucinate math and code. Always run the code yourself and double-check calculations.
3. Beware the "Illusion of Competence": Reading a clear AI explanation does not mean you know it. Close the chat and try to reproduce the solution from memory.
4. Uphold Academic Integrity: Use AI to *learn* the material, debug your own work, or brainstorm. Never use it to generate submissions for graded assignments without explicit professor approval.

Conclusion

Generative AI is a 24/7 personalized tutor. By shifting your prompts from "*solve this for me*" to "*help me understand how to solve this*," you can drastically reduce your time-to-competence and build a deeper, more intuitive mastery of Probability, DBMS, and Software Engineering.

Model 3: Grok 4.5: (DeepSearch)

Using Generative AI as a University Student to Learn Core Subjects

Generative AI tools (ChatGPT, Claude, Gemini, Grok, Copilot, etc.) can act as a 24/7 personal tutor, explainer, practice generator, and code assistant. Below is a practical guide for three common computer science subjects.

1. Probability and Statistics

- Ask for clear, step-by-step explanations of concepts (Bayes' theorem, distributions, hypothesis testing, confidence intervals) in simple language or with real-life analogies.
- Generate unlimited practice problems with solutions and detailed workings. Request variations at easy, medium, and hard levels.
- Upload or paste exam-style questions and ask the AI to solve them while explaining every step.
- Request visual explanations (describe graphs, PDFs, CDFs) or ask for Python/R code to simulate distributions and verify results.
- Use it to check your own solutions and point out common mistakes.

2. Database Management Systems (DBMS)

- Get clear explanations of concepts (normalization, ACID properties, indexing, transactions, ER diagrams, relational algebra).
- Practice writing SQL queries: describe a schema and ask the AI to generate queries, then modify or debug them.
- Request sample schemas, insert data, and create exercises for joins, subqueries, views, stored procedures, and triggers.
- Ask for comparisons between SQL and NoSQL, or between different database systems (MySQL, PostgreSQL, MongoDB).
- Convert natural language requirements into ER diagrams or relational schemas and get feedback on your designs.

3. Software Engineering

- Clarify software development life cycle (SDLC) models, Agile, Scrum, design patterns, UML diagrams, testing strategies, and requirements engineering.

- Generate example use cases, user stories, class diagrams, sequence diagrams, or architecture descriptions from a simple project idea.
- Practice code reviews: paste your code and ask for improvements in readability, modularity, and design principles (SOLID, DRY).
- Request sample test cases (unit, integration, system) or ask how to apply design patterns to a given problem.
- Use it to draft project documentation, SRS documents, or to explain trade-offs between different architectural styles. General Tips for Effective Use
- Always verify AI answers against textbooks, lecture notes, or official documentation — AI can occasionally be incorrect or incomplete.
- Prompt specifically: “Explain like I’m a second-year CS student”, “Give me 5 practice problems with solutions”, or “Show the SQL and the expected output”.
- Combine AI with active learning: try solving problems yourself first, then use AI for hints or verification.
- Use it for revision: ask for summary sheets, flashcards, or “most important concepts for the exam”.
- Be ethical: do not submit AI-generated assignments as your own work. Use it only as a learning aid.

When used actively and critically, generative AI can significantly speed up understanding, provide unlimited practice, and help bridge gaps between theory and application in these subjects.

Key Useful Findings

- **Active Learning Over Direct Answers:** Asking the AI to act as a tutor and guide you step-by-step builds real understanding instead of passive memorization.
- **Code as a Visual Tool:** Using AI to generate quick Python scripts (matplotlib, pandas) helps turn abstract statistics into visual, easy-to-understand graphs.
- **Structured Logic for DBMS:** AI is especially effective at breaking down complex database normalization (1NF to 3NF) and writing efficient SQL queries.
- **Visible Chain-of-Thought:** Watching an AI’s internal reasoning process helps students learn how to systematically break down and debug technical problems.

Quick Tool Comparison

- **DeepSeek-R1 (DeepThink):** Best for formal mathematical proofs, logic, and deep problem-solving.
- **Qwen (Thinking):** Best for clean formatting, study guides, and structured explanations.
- **Grok (DeepSearch):** Best for real-time web research and finding current industry software trends.

Verdict: Best Tool Winner: DeepSeek-R1 (DeepThink)

Reason: For university-level math, databases, and software engineering, understanding the underlying logic is what matters most. DeepSeek-R1's visible reasoning trace shows every step of its work, including how it catches and corrects its own mistakes making it the most effective tool for actual learning.